

Query Answering over Linked Data

Project Report — Semantic Web

Raphaël BINDA

Mathieu WAHARTE

May 2026

Abstract

This report explores query answering over Linked Data using SPARQL and federated queries across publicly available knowledge graphs. We first demonstrate the expressive power of the Semantic Web stack by answering a biomedical question (proteins involved in signal transduction and related to pyramidal neurons) that cannot be handled by large language models alone due to their lack of real-time, structured biological data. We then propose and answer a complex question about COVID-19 vaccine developers using Wikidata. Finally, we discuss how **CONSTRUCT** queries provide formal justification for Linked Data answers, and how hybrid Linked Data / LLM pipelines combine the complementary strengths of both approaches.

Contents

1	Introduction	3
2	Problem Statement	4
3	Question 1: Biomedical Federated Query	5
4	Question 2: COVID-19 Vaccines and Their Developers	7
5	Question 3: Explanation, Justification, and Hybrid Approach	9
5.1	Justifying answers with CONSTRUCT	9
5.2	How the hybrid Linked Data + LLM approach works	10
6	Discussion	12
6.1	Benefits of Linked Data	12
6.2	Limitations of Linked Data	12
6.3	Benefits and limitations of the hybrid approach	12
7	Conclusion	13

1 Introduction

The World Wide Web, originally designed for human use, has evolved towards a machine-readable layer known as the *Semantic Web* or *Web of Data* [1]. At its core, the Semantic Web relies on the Resource Description Framework (RDF) to express knowledge as triples (*subject–predicate–object*), and on SPARQL as the standard query language to retrieve and manipulate that knowledge. The Linked Open Data (LOD) cloud [3] today contains billions of such triples distributed across hundreds of interconnected datasets, covering domains from biomedicine (UniProt, Bgee, LinkedLifeData) to culture and encyclopaedic knowledge (DBpedia, Wikidata).

In his 2009 TED talk, Tim Berners-Lee challenged the scientific community to make use of Linked Data for answering complex, cross-domain questions [2]. One such question, “*What proteins are involved in signal transduction and also related to pyramidal neurons?*”, illustrates precisely where the Semantic Web shines: it requires joining heterogeneous, specialised databases in a single, structured, reproducible query, something that Large Language Models (LLMs) cannot reliably do.

This project addresses three tasks:

- Q1.** Demonstrate a Semantic Web method to answer the above biomedical question using federated SPARQL.
- Q2.** Propose and answer a new complex question using Wikidata: which organisations developed COVID-19 vaccines, and in which countries are they based?
- Q3.** Explain and justify the answers using CONSTRUCT queries and discuss a hybrid Linked Data + LLM approach.

2 Problem Statement

Answering complex factual questions requires three capabilities that simple keyword search or LLMs lack: (i) **access to up-to-date, structured data**, (ii) **the ability to join information across heterogeneous sources**, and (iii) **provenance and justification** of the answer.

LLMs like ChatGPT encode world knowledge statically at training time. They cannot query live databases, cannot guarantee that their answers reflect the current state of specialised repositories (e.g. UniProt protein annotations), and cannot easily provide a formal justification (a *proof*) of how an answer was derived from source data.

The Semantic Web stack directly addresses these limitations (and also predates LLMs). The challenge is to translate natural-language questions into correct SPARQL queries, navigate the data models of multiple endpoints, and combine the results coherently. For these steps, LLMs can be used as assistants.

3 Question 1: Biomedical Federated Query

We start with the question posed by Tim Berners-Lee, requiring connecting biological databases:

What proteins are involved in signal transduction and also related to pyramidal neurons?

First, we identify the relevant datasets that contain the necessary information:

- **UniProt** (<https://sparql.uniprot.org/sparql>): the reference database for protein sequences and functional annotation. Each protein can be annotated with Gene Ontology (GO) terms, including GO:0007165 (*signal transduction*).
- **Bgee** (<https://www.bgee.org/sparql>): a database of gene expression across anatomical entities and developmental stages. It provides expression data for genes in specific cell types such as pyramidal neurons.
- **Gene Ontology (OBO)** (<https://purl.obolibrary.org/obo/>): the controlled vocabulary linking biological processes to GO terms, used to navigate the hierarchy of signal transduction subprocesses.

We also explored other datasets but they didn't provide the necessary data or were difficult to query.

The key insight is that the two conditions in the question are held in *different* databases:

1. Signal transduction annotation lives in UniProt (via GO terms).
2. Expression in pyramidal neurons lives in Bgee (via anatomical entity labels).

To query both services in a single query, we found the **SERVICE** keyword that can delegate sub-queries to remote endpoints and join the results locally. The bridge between the two databases is the UniProt identifier, which Bgee exposes through the `lscr:xrefUniprot` predicate.

We therefore propose the following query:

```

1 PREFIX up:      <http://purl.uniprot.org/core/>
2 PREFIX go:      <http://purl.obolibrary.org/obo/>
3 PREFIX lscr:    <http://purl.org/lscr#>
4 PREFIX genex:   <http://purl.org/genex#>
5 PREFIX orth:    <http://purl.org/net/orth#>
6 PREFIX rdfs:    <http://www.w3.org/2000/01/rdf-schema#>
7
8 SELECT DISTINCT ?protein ?proteinName ?goTerm ?goLabel WHERE {
9
10   # Condition 1: Protein involved in signal transduction (
11     UniProt)
12   SERVICE <https://sparql.uniprot.org/sparql> {
13     ?protein a up:Protein ;
14             up:recommendedName/up:fullName ?proteinName ;

```

```

14         up:classifiedWith ?goTerm .
15     ?goTerm rdfs:subClassOf* go:GO_0007165 .    # signal
16         transduction
17     ?goTerm rdfs:label ?goLabel .
18 }
19 # Condition 2: Gene expressed in pyramidal neurons (Bgee)
20 SERVICE <https://www.bgee.org/sparql> {
21     ?gene a orth:Gene ;
22         genex:isExpressedIn ?cond .
23     ?cond genex:hasAnatomicalEntity ?anatEntity .
24     ?anatEntity rdfs:label "pyramidal neuron"@en .
25     ?gene lscr:xrefUniprot ?protein .
26 }
27 }

```

Listing 1: Federated SPARQL query joining UniProt and Bgee.

We faced several challenges in formulating and executing this federated query:

- **Data model exploration:** the predicate connecting a protein to an anatomical entity is not obvious. We first had to inspect the Bgee ontology to discover the `genex:isExpressedIn` / `genex:hasAnatomicalEntity` chain.
- **GO hierarchy:** signal transduction has many sub-processes. Using `rdfs:subClassOf*` (transitive closure) correctly captures all relevant GO terms without hardcoding each one.
- **Cross-database identifier alignment:** UniProt protein URIs must match Bgee's `lscr:xrefUniprot` values. This required checking that both sides use the canonical UniProt accession format.

4 Question 2: COVID-19 Vaccines and Their Developers

The complex question we propose is:

Which COVID-19 vaccines were developed by which organisations, and in which country are those organisations based?

After some exploration, we identified that the necessary information is available in **Wiki-data** (<https://query.wikidata.org>). Vaccine entries are linked to their developers via the `wdt:P178` (*developer*) property, and developers are linked to their country of headquarters via `wdt:P17` (*country*). The class `wd:Q84263196` identifies COVID-19 vaccines.

Therefore, we propose the following SPARQL query to retrieve the desired information:

```

1 PREFIX wd:          <http://www.wikidata.org/entity/>
2 PREFIX wdt:         <http://www.wikidata.org/prop/direct/>
3 PREFIX wikibase:    <http://wikiba.se/ontology#>
4 PREFIX bd:          <http://www.bigdata.com/rdf#>
5
6 SELECT DISTINCT ?vaccine ?vaccineLabel
7                 ?developer ?developerLabel
8                 ?country ?countryLabel
9 WHERE {
10   ?vaccine wdt:P1924 wd:Q84263196 ;      # instance of: COVID-19
11         vaccine
12         wdt:P178 ?developer .           # developer
13   ?developer wdt:P17 ?country .         # country
14
15   SERVICE wikibase:label {
16     bd:serviceParam wikibase:language "en" .
17   }
18 ORDER BY ?countryLabel ?vaccineLabel

```

Listing 2: Wikidata SPARQL query for COVID-19 vaccine developers.

This query returns a list of COVID-19 vaccines, their developers, and their countries as such:

Vaccine	Developer	Country
CoronaVac	Sinovac Biotech	China
ArVac Cecilia G.	UNSAM / CONICET	Argentina
COVAX-19	Flinders U. / Vaxine	Australia
Ad26.COV2.S	Janssen Pharmaceutica	Belgium
Comirnaty (BNT162b2)	BioNTech / Pfizer	Germany / USA
Oxford–AstraZeneca	University of Oxford / AZ	United Kingdom
Sputnik V	Gamaleya Institute	Russia

Table 1: A selection of results returned by the Wikidata query.

Mixing LLMs and Semantic Web, ChatGPT was used to *suggest* which Wikidata properties and item identifiers to use (e.g. P1924, Q84263196), which significantly reduced the time spent browsing the Wikidata property catalogue. However, the actual, structured, and up-to-date answer was retrieved exclusively from Wikidata through SPARQL, ChatGPT alone would risk returning outdated or fabricated vaccine–country associations.

5 Question 3: Explanation, Justification, and Hybrid Approach

5.1 Justifying answers with CONSTRUCT

The `SELECT` form of SPARQL returns tabular results. To *explain* an answer (exposing the exact RDF triples that support it), SPARQL’s `CONSTRUCT` form builds an explicit sub-graph. This sub-graph constitutes a machine-readable *proof* or justification.

For Question 2, the following `CONSTRUCT` query returns the RDF graph that justifies each vaccine–developer–country answer:

```

1 PREFIX wd: <http://www.wikidata.org/entity/>
2 PREFIX wdt: <http://www.wikidata.org/prop/direct/>
3 PREFIX wikibase: <http://wikiba.se/ontology#>
4 PREFIX bd: <http://www.bigdata.com/rdf#>
5 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
6
7 CONSTRUCT {
8     ?vaccine wdt:P1924 wd:Q84263196 ;
9             wdt:P178 ?developer ;
10            rdfs:label ?vaccineLabel .
11
12     ?developer wdt:P17 ?country ;
13             rdfs:label ?developerLabel .
14
15     ?country rdfs:label ?countryLabel .
16 }
17 WHERE {
18     ?vaccine wdt:P1924 wd:Q84263196 ;
19             wdt:P178 ?developer .
20     ?developer wdt:P17 ?country .
21
22     SERVICE wikibase:label { bd:serviceParam wikibase:language "
23         en". }
24 }
```

Listing 3: `CONSTRUCT` query for Q2 result visualization.

This query returns an explicit RDF graph where the interconnections between vaccines, their developers, and countries are visualized as named triples. Figure 1 shows the resulting graph structure.

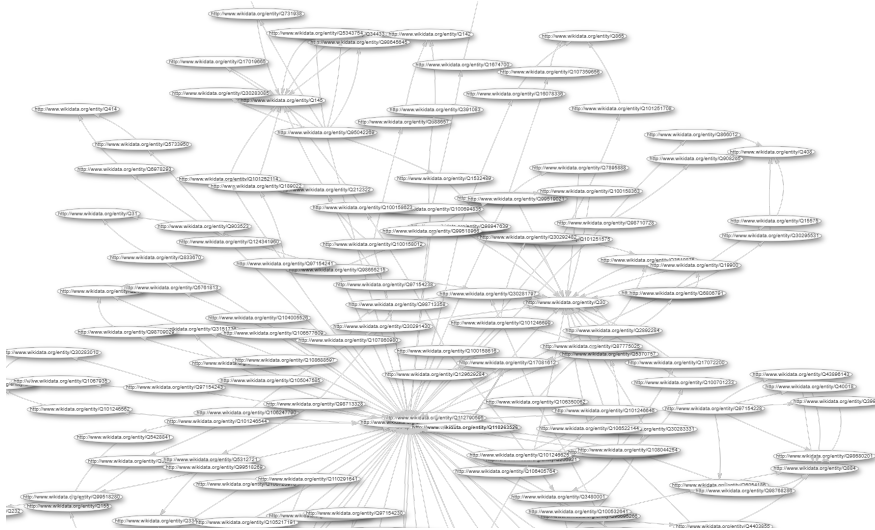


Figure 1: RDF graph visualization of vaccine-developer-country relationships returned by the CONSTRUCT query in Listing 3.

5.2 How the hybrid Linked Data + LLM approach works

Neither Linked Data nor LLMs are sufficient alone. Table 2 summarises their complementary strengths.

Linked Data	LLMs
Structured, machine-readable data	Natural language understanding
Up-to-date	Static knowledge (training cut-off)
Formally provable answers	No formal provenance
Requires precise query formulation	Handles vague / ambiguous questions
Difficult for non-expert users	Accessible to everyone
Can join multiple databases	Limited to training data

Table 2: Comparison of Linked Data and LLM capabilities.

A practical pipeline proceeds as follows:

1. **Intent parsing (LLM)**: the natural language question is given to an LLM, which identifies entities (“COVID-19 vaccine”, “developer”, “country”), relevant ontology terms, and suggests candidate SPARQL triple patterns and property identifiers.
2. **Query generation (LLM + human validation)**: the LLM generates a draft SPARQL query. The user or an automated validator checks that the properties and classes exist in the target knowledge graph.
3. **Execution (Linked Data)**: the validated query is executed against the live SPARQL endpoint (Wikidata, UniProt, Bgee, etc.) and returns structured, up-to-date results.

4. **Answer generation (LLM)**: the tabular results are fed back to the LLM, which summarises them in fluent natural language and highlights notable patterns (e.g. “Most approved vaccines were developed in partnership between a pharmaceutical company and a university”).
5. **Justification (CONSTRUCT)**: optionally, a **CONSTRUCT** query retrieves the supporting RDF sub-graph, which is shown to the user as a provenance certificate.

This pipeline mirrors emerging architectures such as *Retrieval-Augmented Generation* (RAG), applied here to structured knowledge graphs rather than unstructured text corpora.

6 Discussion

6.1 Benefits of Linked Data

- **Federated access:** a single SPARQL query can join data from UniProt, Bgee, and GO without pre-downloading or merging the datasets.
- **Open standards:** RDF and SPARQL are W3C standards, ensuring interoperability and long-term stability.
- **Provenance:** CONSTRUCT queries make justification first-class, enabling auditability.
- **Precision:** results are exact matches to the query, not probabilistic generations.

6.2 Limitations of Linked Data

- **Steep learning curve:** formulating correct SPARQL queries requires knowledge of the target ontologies and data models.
- **Endpoint fragility:** federated queries fail if any remote endpoint is unavailable or slow.
- **Data completeness:** Wikidata and other community-driven graphs may have missing or inconsistent entries.
- **No reasoning:** Linked Data cannot interpret documents, images, or unstructured annotations without additional NLP.

6.3 Benefits and limitations of the hybrid approach

The hybrid pipeline inherits the benefits of both paradigms: the accessibility of LLMs and the precision and provenance of Linked Data. Its main limitation is error propagation: an incorrectly generated SPARQL query (hallucinated property names, wrong class URIs) will produce wrong or empty results with no obvious error message for the end user. Human-in-the-loop validation at the query generation step is therefore essential.

7 Conclusion

This project demonstrated that the Semantic Web stack remains uniquely suited for answering complex, cross-domain factual questions that require structured, live, and justified data retrieval. Federated SPARQL queries, spanning UniProt, Bgee, and Wikidata, provide answers that LLMs cannot reliably generate from training data alone. The **CONSTRUCT** mechanism adds formal explainability, a critical feature for scientific and high-stakes applications. Finally, the hybrid Linked Data + LLM pipeline offers a pragmatic path forward: LLMs lower the barrier to query formulation, while Linked Data guarantees accuracy, freshness, and provenance.

References

- [1] T. Berners-Lee, J. Hendler, and O. Lassila, “The Semantic Web,” *Scientific American*, vol. 284, no. 5, pp. 34–43, 2001.
- [2] T. Berners-Lee, “The Next Web” (TED Talk), 2009. https://www.ted.com/talks/tim_berniers_lee_the_next_web
- [3] M. Schmachtenberg, C. Bizer, and H. Paulheim, “Adoption of the Linked Data Best Practices in Different Topical Domains,” in *Proc. ISWC*, 2014. <https://lod-cloud.net>
- [4] S. Harris, A. Seaborne, et al., “SPARQL 1.1 Query Language,” W3C Recommendation, March 2013. <https://www.w3.org/TR/sparql11-query/>
- [5] UniProt Consortium, “UniProt: the Universal Protein Knowledgebase in 2023,” *Nucleic Acids Research*, vol. 51, D1, pp. D523–D531, 2023. <https://sparql.uniprot.org/sparql>
- [6] F. Bastian et al., “Bgee in 2023: Improved coverage and new features for gene expression data in animals,” *Nucleic Acids Research*, vol. 51, D1, pp. D1951–D1958, 2023. <https://www.bgee.org/sparql>
- [7] D. Vrandečić and M. Krötzsch, “Wikidata: A Free Collaborative Knowledgebase,” *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014. <https://query.wikidata.org>
- [8] Ontotext, “Linked Life Data,” <https://linkedlifedata.com>, 2023.